

AI Audit Report — HW05 Performance Testing

- **Sinh viên:** Lê Nhựt Duy — **MSSV:** 23127178
- **Bài:** HW05-AI Performance Testing — **AI Policy:** Open (§9 bắt buộc có phụ lục này)

I use AI tools for the following tasks.

AI tool	Model	Dùng cho
Claude Code	Opus 5 (1M context)	Chọn phạm vi endpoint (§5) · dựng repo + tooling · sinh 4 test plan JMeter · bản mirror k6 · phân tích <code>.jtl</code> · viết báo cáo

§2 đòi dùng AI "*step by step, not with a single generic prompt*". File này ghi **prompt nguyên văn của sinh viên** — và những prompt đó ngắn. Bảy bước của quy trình được ghi riêng ở `design-log.md`: mỗi bước hỏi gì, căn cứ nào, quyết ra sao, thay đổi cụ thể nào trong file. Hai file tách nhau **có chủ ý** — trộn lại thành "sinh viên đã hỏi bảy câu" sẽ là bịa, và §11 cấm đúng điều đó.

Ranh giới cần đọc trước khi chấm mục "human review": các trường AI sai / bỏ sót dưới đây ghi lại những gì **phát hiện được trong phiên làm việc** (phần lớn qua smoke test và qua đọc lại code SUT). Trường **Human review** là phần tôi tự kiểm và tự chịu trách nhiệm.

Mỗi trường Human review dùng **hai nhãn**, cố tình tách nhau:

- **(SV đã kiểm)** — việc tôi thực sự tự làm trong phiên, kiểm lại được bằng chứng cứ có trong repo (ảnh chụp, dòng trong `run-log.md`, output script).
- **(SV chưa tự kiểm)** — chỗ tôi **nhận theo bằng chứng AI đưa ra** mà không kiểm độc lập, kèm việc cụ thể lẽ ra phải làm để kiểm.

Ghi nhãn thứ hai là có chủ ý. Viết cả 12 lượt thành "đã kiểm hết" thì nhanh hơn, nhưng đó đúng là loại bằng chứng dựng mà §11 phạt — và một bản audit không có dòng nào tự nhận là chưa kiểm thì tự nó đáng ngờ.

Các lượt tương tác

Interaction #1 — Chọn phạm vi endpoint (§5)

- **Task / Scenario:** §5 — chọn endpoint group, chống trùng trong nhóm
- **Bước trong quy trình:** trước `perf-test-plan` bước 1
- **AI tool:** Claude Code (Opus 5)
- **Date & time:** 2026-08-13 11:05
- **Prompt:** ngoài endpoint nhóm đã chọn thì bạn hãy chọn cho tui 1 vài endpoint đi (kèm 2 ảnh chụp chat nhóm: đăng ký endpoint của Nguyễn, Quan, Thế Đạt; kèm đề bài `2026.HW05.Performance Testing_En.pdf`)

- **AI output (tóm tắt):** Đọc 31 route thật trong `backend/server.js`, đối chiếu với đăng ký của 3 thành viên, kết luận toàn bộ luồng khách hàng đã bị chiếm → đề xuất workflow **admin back-office** 5 bước (`login` → `admin/orders` → `admin/users` → `import-products` → `orders/:id/status`) phủ đủ 3 nhóm. Kèm 5 endpoint bổ sung còn trống và 3 lưu ý về SUT (lockout kích hoạt sau 2 lần sai; không có rate limiting; `import-products` báo sai số dòng).
- **AI sai / bỏ sót: Một kết luận sai do chỉ đọc code, không chạy thử.** AI khẳng định `POST /api/admin/import-products` "báo số dòng đã insert nhỏ hơn thực tế" vì `stmt.finalize()` trả response trước các callback `stmt.run` (`server.js:234`). Kiểm bằng request thật ở lượt #8 thì **sai hoàn toàn**: 5/5, 60/60, và 2/3 khi có dòng thiếu `name` — đều đúng, vì `node-sqlite3` xếp các lệnh trên cùng handle theo thứ tự và callback của `finalize` chạy sau chúng. Kết luận này đã lan vào 5 file tài liệu trước khi bị bắt.
- **Vì sao bỏ sót: chất lượng prompt và phương pháp** — prompt hỏi "đọc code và đề xuất endpoint", nên AI trả lời bằng suy luận từ code và trình bày suy luận đó như một sự thật đã kiểm. Không có bước nào bắt nó phân biệt "đọc được từ code" với "đã chạy thử".
- **Human review: (SV đã kiểm)** Phần chống trùng §5 tôi đối chiếu được trực tiếp, vì **bảng đăng ký là do tôi cung cấp**: hai ảnh chat nhóm (Nguyễn, Quan, Thế Đạt) và file đề bài là tôi gửi vào phiên. Tôi soát từng endpoint của workflow admin back-office với ba danh sách đã đăng ký — không endpoint nào trùng, và toàn bộ luồng khách hàng (`products`, `cart`, `checkout`, `apply-coupon`) đã bị ba bạn chiếm nên tôi chấp nhận hướng admin. Bảng này chép vào `report/main-report.md §1.1` để nó nằm trong phần được nộp. **(SV chưa tự kiểm)** 5 dòng `server.js` được dẫn tôi không tự mở đọc lúc đó. Đúng chỗ đó hỏng: nhận định `import-products` "báo sai số dòng insert" **có kèm số dòng code** nên tôi tin, và nó sai — lượt #8 bác bỏ bằng request thật. Bài học tôi lấy về: **dẫn được số dòng code không làm một nhận định trở thành đã kiểm**, và từ lượt đó tôi bắt đầu hỏi "cái này chạy thử chưa".
- **Commit:** `docs: playbook, agent skills and report scaffolding`

Interaction #2 — Dựng repo và tooling

- **Task / Scenario:** hạ tầng bài làm
- **Bước trong quy trình:** trước `perf-test-plan` bước 1
- **AI tool:** Claude Code (Opus 5)
- **Date & time:** 2026-08-13 11:10
- **Prompt:** tham khảo các bài hw trc setup repo cho HW05 giùm tui
- **AI output (tóm tắt):** Đọc quy ước HW02/03/04 (`.gitignore`, `README self-assessment`, `tools/`, `.claude/skills/`), dựng repo HW05 cùng bộ khung + 9 script + 4 Agent Skill + 3 tài liệu docs + khung 5 tài liệu §14.
- **AI sai / bỏ sót: Một bug thật trong script do AI viết:** `preflight.mjs` đọc `/usr/libexec/java_home -V` từ `stdout`, nhưng lệnh này in ra `stderr` → script báo `[OK] Java (arm64)` với giá trị **rong**, tức là một lỗi im lặng: báo xanh trong khi không kiểm được gì. Đã sửa bằng hàm `shell()` gộp `2>&1` và đọc `os.arch` từ chính JDK.
- **Vì sao bỏ sót:** đặc điểm của công cụ — hành vi `stdout/stderr` của `java_home` không suy ra được từ ngữ nghĩa lệnh, phải chạy thử mới thấy.
- **Human review: (SV đã kiểm)** Chuẩn so sánh là do tôi đặt, không phải AI tự chọn: tôi yêu cầu dựng theo **HW01/HW02** — hai bài tôi được chấm cao nhất — nên repo phải có `README` với bảng `self-assessment`, `tools/` chứa script chạy lại được, `git-log/` có log commit. Sau đó tôi còn yêu cầu **dựng riêng folder submit** để biết chính xác mình nộp gì, và chốt là **sửa thì sửa cả hai bản. (SV**

chưa tự kiểm) Bug stdout/stderr của `java_home` tôi không tự phát hiện được — đó là loại lỗi tôi sẽ tin ngay, vì script in `[OK]` Java màu xanh. Tôi chỉ xác nhận được **hệ quả sau khi sửa**: preflight chỉ đúng ra `java` trên PATH là Temurin 8 **x86_64** trong khi máy có Temurin 26 arm64 — và điều đáng nhớ là **bản báo xanh sai và bản báo đúng nhìn gần như y nhau**, khác nhau ở một giá trị rỗng mà tôi không nhìn ra.

- **Commit:** `test: environment checks and data-driven fixtures`

Interaction #3 — Kế hoạch thực hiện

- **Task / Scenario:** lập kế hoạch
- **Bước trong quy trình:** —
- **AI tool:** Claude Code (Opus 5)
- **Date & time:** 2026-08-13 11:40
- **Prompt:** `giờ tui phải làm j nữa`
- **AI output (tóm tắt):** Danh sách 8 bước theo thứ tự ưu tiên theo trọng số điểm (Task 1 = 60/100 nên làm trước), kèm lệnh cụ thể cho từng bước.
- **AI sai / bỏ sót:** —
- **Vì sao bỏ sót:** —
- **Human review: (SV đã kiểm)** Đây là lượt tôi chủ động nhất, và tôi lặp lại nó **năm lần** ở năm mốc khác nhau của bài ("`giờ tui phải làm j nữa`" → ... → "`là giờ tui còn j nữa`"). Lý do lặp: tôi không muốn một kế hoạch dài đọc một lần rồi trôi, tôi muốn **danh sách ngắn còn lại bao nhiêu việc** ở mỗi mốc, xếp theo trọng số điểm — Task 1 chiếm 60/100 nên phải xong trước Task 2/3. Cách kiểm coverage của tôi là **trích nguyên văn từng yêu cầu trong đề rồi hỏi nó nằm ở đâu**: "*Use three different report views — cái này nằm ở đâu ak*", "*cái này bạn run chưa*" (endurance), "*cái này thì mà chưa chạy đúng hok do chưa thấy cap màn hình mà*". Ba câu đó tìm ra ba lỗi khác nhau, trong đó câu thứ ba là lỗi thật (xem lượt #7). **(SV chưa tự kiểm)** Bản thân **thứ tự 8 bước** tôi nhận theo lập luận trọng số điểm mà không tự dò lại toàn bộ 17 mục của đề — nên tôi bù bằng cách trích yêu cầu ra hỏi lẻ như trên, và cuối bài chốt lại bằng `TASKS.md` map từng yêu cầu → file + mục.
- **Commit:** —

Interaction #4 — Sinh 4 test plan và chạy

- **Task / Scenario:** Load · Stress · Spike · Soak
- **Bước trong quy trình:** `perf-test-plan` bước 1→7
- **AI tool:** Claude Code (Opus 5)
- **Date & time:** 2026-08-13 12:00
- **Prompt:** `hãy thực hiện toàn bộ giúp tui`
- **AI output (tóm tắt):** Cài JMeter, seed 50 tài khoản + 430 order + 3 file CSV, viết `tools/gen-test-plans.py` sinh 4 `.jmx` từ **một** định nghĩa workflow dùng chung (§6 đòi 3 plan cùng workflow), chạy smoke test 20s, sửa lỗi, rồi chạy 3 lượt chính + soak.
- **AI sai / bỏ sót:** Hai lỗi thật trong test plan do AI sinh, smoke test mới lộ ra: 1. **Nhánh lockout bị tính thành lỗi hiệu năng.** AI gắn assertion regex `40[13]` cho request đăng nhập sai mật khẩu và tưởng thế là đủ. Nhưng JMeter mặc định coi mọi 4xx là Fail, và assertion chỉ **thêm** được lỗi chứ không **xoá** được cờ Fail. Kết quả smoke test: **41,38% error** trong khi SUT hoạt động hoàn toàn đúng. Sửa bằng `JSR223PostProcessor` (Groovy) gọi `prev.setSuccessful(true)` khi mã trả về khớp `40[13]`. 2. **Bước 5 assert cứng HTTP 200, không tính tới state machine.** PUT

/api/admin/orders/:id/status đi qua FR-10 (server.js:537-551): một order chỉ chuyển tiếp được **một lần** cho mỗi trạng thái, nên từ lần lặp thứ hai trở đi SUT trả 400 "invalid transition" — đúng đặc tả. AI còn xen kẽ `shipping` trong `orders.csv`, mà từ `pending` thì `shipping` là chuyển đổi **không hợp lệ** → một nửa sample của bước 5 trả 400 ngay từ đầu. Sửa: `assert 200|400` kèm giải thích, `orders.csv` chỉ dùng `confirmed`, và thêm `tools/reset-orders.mjs` đưa order về `pending` trước mỗi lượt.

- **Vì sao bỏ sót:** cả hai đều thuộc nhóm **đặc điểm của endpoint**, không phải giới hạn model: hành vi "assertion không xoá được cờ Fail" là chi tiết của JMeter chỉ hiện ra khi chạy, và state machine FR-10 nằm trong code SUT chứ không nằm trong đặc tả API. Prompt cũng chưa nêu hai điều này — nên có phần lỗi ở chất lượng prompt.
- **Human review: (SV đã kiểm)** Tôi không nhận 4 file `.jmx` như hộp đen — tôi hỏi lại **hai lần** ở hai thời điểm: "*mấy file jmx ntn ak*" rồi sau đó với bản trong folder submit "*tui vẫn chưa hìu mấy file này để làm j*". Lần thứ hai là vì tôi muốn biết file trong bản nộp có **đúng là file đã chạy** hay chỉ là bản copy. Tôi cũng tự kiểm một yêu cầu cụ thể của §6: "*Use three different report views — cái này nằm ở đâu ak*" → xác nhận Load/Soak dùng **Summary Report**, Stress dùng **Aggregate Report**, Spike dùng **View Results Tree**, ba listener khác nhau nằm thật trong XML của ba plan chứ không chỉ được nhắc trong báo cáo. **(SV chưa tự kiểm)** Hai lỗi assertion (cờ Fail của 4xx, state machine FR-10) thì tôi **không đủ nền JMeter để bắt trước khi chạy** — chúng lộ ra nhờ **smoke test 20 giây chạy trước lượt 6 phút**, và đó là chỗ tôi thấy giá trị của bước đó: nếu bỏ smoke test thì đã có 3 lượt chính với error rate vô nghĩa, và tôi sẽ **không biết** là nó vô nghĩa vì plan vẫn chạy xong, vẫn ra dashboard đẹp.
- **Commit:** `test(plans): generate the four JMeter plans from one shared workflow`

Interaction #5 — Hai lỗi nữa chỉ lượt chạy dài mới lộ ra

- **Task / Scenario:** Load (lượt chạy thật đầu tiên, đã **huỷ giữa lượt**)
- **Bước trong quy trình:** `perf-test-plan` bước 6→7 (chạy và đọc kết quả trước khi tin nó)
- **AI tool:** Claude Code (Opus 5)
- **Date & time:** 2026-08-13 12:21
- **Prompt:** cùng lượt #4 — đây là phần đọc kết quả và sửa, không phải prompt mới.
- **AI output (tóm tắt):** Lượt Load chạy được, nhưng error rate đứng ở ~2,9% và throughput chỉ ~10 sample/s. Phân tích `.jtl` đang ghi cho thấy hai nguyên nhân độc lập nhau.
- **AI sai / bỏ sót: 3. Dữ liệu test tự đầu độc lượt chạy.** `data/users.csv` do AI sinh có 2 dòng cuối dùng `WrongPassword!` (ý ban đầu là để test lockout). Nhưng thread group chính đọc chính file đó với `recycle=true`, nên nó cũng gặp 2 dòng ấy → login 401 → **hai tài khoản bị khoá 180s** → mọi lần đọc lại 2 dòng đó trả 403, và vì không có token nên **cả 4 bước còn lại của iteration đó cũng 403**. Đó chính là toàn bộ ~2,9% error: đúng 9 lần × 5 label. Không một sample nào trong số đó nói gì về hiệu năng của SUT. Sửa: `users.csv` chỉ còn tài khoản đăng nhập được; nhánh lockout tách sang `data/users_lockout.csv`.
- **4. Think-time nhân 5 lần so với ý định.** AI đặt Uniform Random Timer 1000–3000ms ở scope **thread group**, mà JMeter chèn timer trước **từng sampler** — 5 lần một iteration, tức 5–15 giây/iteration thay vì 1–3s. Hệ quả: 20 VU sinh ra ~10 sample/s thay vì ~60, nên "Load test 20 VU" thực chất đo một mức tải nhẹ hơn nhiều lần so với thiết kế. Sửa: chia lại còn 200–600ms mỗi bước để **tổng** mỗi iteration đúng 1–3s, giữ nguyên ngữ nghĩa "admin dừng giữa các màn hình".
- **Vì sao bỏ sót:** lỗi 3 là **chất lượng prompt** — không ai nói rõ "file dữ liệu của luồng chính không được chứa dòng âm tính". Lỗi 4 là **đặc điểm công cụ:** phạm vi tác dụng của timer trong JMeter là

kiến thức phải có sẵn, và AI đã áp dụng sai một cách hoàn toàn im lặng — plan vẫn chạy, vẫn ra số, chỉ là đo sai mức tải.

- **Human review: (SV đã kiểm)** Việc **xoá sạch** lượt chạy này thì tôi kiểm được, và tôi có kiểm: mở `results/run-log.md` đếm — chỉ còn **12 dòng thuộc 3 batch được giữ**, không có dòng nào của lượt bị huỷ, và trong `results/` không còn `.jtl` hay folder dashboard nào của nó. Tôi chốt nguyên tắc này cho cả bài: **số liệu từ một lượt đã biết là sai thì không xuất hiện ở đâu trong bài, kể cả kèm chú thích** — vì một bảng có chú thích "cột này không dùng được" vẫn sẽ bị ai đó đọc và trích. Sau đó tôi áp đúng nguyên tắc đó thêm hai lần: xoá batch 14/08 khi có batch sạch thay thế, và xoá một lượt Stress bị ngắt ở giây thứ 51. **(SV chưa tự kiểm)** Hai lỗi ở lượt này tôi không bắt được, và **cả hai đều là loại không báo lỗi**: `users.csv` tự khoá tài khoản của chính nó, và think-time bị nhân 5 vì timer sai scope. Cái thứ hai đáng sợ hơn với tôi — plan vẫn ghi "20 VU", vẫn chạy hết 6 phút, chỉ là **mức tải thật nhẹ hơn thiết kế 5 lần**, và không có con số nào trong dashboard tự nói ra điều đó.
- **Commit:** `test(plans): stop the test data from poisoning its own run`

Interaction #6 — Cùng một lỗi, lần thứ hai

- **Task / Scenario:** Load (lượt chạy thật thứ hai, cũng **huỷ giữa lượt**)
- **Bước trong quy trình:** `perf-test-plan` bước 7
- **AI tool:** Claude Code (Opus 5)
- **Date & time:** 2026-08-13 13:10
- **Prompt:** cùng lượt #4.
- **AI output (tóm tắt):** Sau khi sửa 4 lỗi trước, lượt Load chạy đúng 50 sample/s như thiết kế và 0% error trong 90 giây đầu. Nhưng error rate rồi leo lên **18,25%**.
- **AI sai / bỏ sót:** 7. **Áp cơ chế đúng cho nhánh lockout mà quên áp cho bước 5.** Toàn bộ 667 sample "lỗi" là `400` của `PUT /api/admin/orders/:id/status` — tức phản hồi **đúng** của state machine FR-10 khi order đã chuyển trạng thái rồi. Assertion `200|400` pass, nhưng JMeter đã gán cờ Fail cho 4xx từ trước và assertion không xoá được cờ đó — **đúng cơ chế đã phát hiện và đã sửa ở lượt #4 cho nhánh lockout**, chỉ là không nghĩ tới việc bước 5 cũng cần. Sửa: dùng lại `mark_expected_4xx("400")` cho bước 5.
- **Vì sao bỏ sót:** không phải giới hạn model cũng không phải đặc điểm endpoint — đây là **suy luận không được mang sang chỗ tương tự**. Đã hiểu đúng cơ chế ở một nơi mà không tự hỏi "chỗ nào khác trong plan cũng trả 4xx hợp lệ?".
- **Human review: (SV đã kiểm)** Tôi kiểm được **kết quả** của lần sửa này: sau đó cả 4 lượt của batch cuối đều **0% error** trên 358.661 sample, và trong `results/summary.md` phần chia theo mã trả về, ~50.000 dòng `400` của bước 5 nằm ở nhóm **đã được đánh dấu là hợp lệ** chứ không bị cộng vào error rate. Con số này tôi tin, vì nó khớp với việc **không có sample nào trả 500**. **(SV chưa tự kiểm)** Tôi không tự phát hiện ra rằng cơ chế đã sửa cho nhánh lockout cần được mang sang bước 5 — đây là **lỗi lần thứ hai của cùng một nguyên nhân**, và nó lọt qua mắt tôi vì tôi đọc "đã sửa lỗi 4xx" như một việc đã xong, không như một cơ chế cần đi soát mọi chỗ tương tự. Chỗ này là bài học tôi giữ cho phần đọc metric: **error rate là chỉ số phải nghi ngờ trước tiên**, vì 18,25% error hiện ra trên một hệ thống hoàn toàn khoẻ, và nếu tin nó thì báo cáo sẽ kết luận sai hẵn về endpoint transactional — đúng loại lỗi tôi liệt kê ở Task 2.
- **Commit:** `test(plans): count the expected 400 from FR-10 as a pass`

Interaction #7 — Lỗi trong chính tooling đo tài nguyên

- **Task / Scenario:** thu bằng chứng tài nguyên (§6)

- **Bước trong quy trình:** resource-evidence bước 2
- **AI tool:** Claude Code (Opus 5)
- **Date & time:** 2026-08-13 13:19
- **Prompt:** cùng lượt #4.
- **AI output (tóm tắt):** tools/sample-resources.sh lấy mẫu CPU/RSS của backend mỗi 2 giây để có số liệu tính được cho endurance threshold.
- **AI sai / bỏ sót: 8. Bám vào tiến trình sai.** Script dùng pgrep -f 'node server.js' | head -1 . Trên máy này pgrep khớp hai pid: tiến trình /bin/zsh -c ... đã khởi động backend (chuỗi đó nằm trong command line của nó) và tiến trình node thật. head -1 chọn cái shell → file CSV ghi **RSS 0,5 MB và CPU 0,0%** suốt 6 phút, trong khi node thật đang ở ~76 MB / ~18% CPU. Sửa: lọc theo token đầu của command line phải là node (ps -Ao pid=,command= | awk '/[n]ode server\.js/' && \$2 ~ /node\$/ ...').
- **Vì sao bỏ sót: đặc điểm môi trường.** pgrep -f khớp trên toàn bộ command line, nên một shell wrapper chứa cùng chuỗi cũng khớp — điều này chỉ hiện ra khi backend được khởi động qua wrapper, đúng như cách phiên làm việc này khởi động nó.
- **Human review: (SV đã kiểm — lượt tôi can thiệp mạnh nhất.)** Ba việc, theo thứ tự:

(a) Tôi bắt lỗi bằng chứng trước khi nó thành lỗi trong bài. Đọc yêu cầu §6 rồi hỏi lại: "cái này thì mày chưa chạy đúng hok do chưa thấy cap màn hình mà" — tức là bài đang khẳng định đã thu bằng chứng tài nguyên nhưng trong repo không có ảnh nào. Số liệu CSV không thay được ảnh, vì §6 đòi ảnh.

(b) Tôi từ chối cách chạy ban đầu, và đây là chỗ tôi thấy quan trọng nhất. Các lượt trước được chạy trong shell nền của AI — tôi không thấy gì, và một ảnh Activity Monitor chụp lúc đó sẽ không có cửa sổ terminal nào của tôi trong đó. Tôi hỏi thẳng: "ủa là sao cho tao script chạy đi chứ rùi terminal là terminal nào". Từ đó đổi sang tôi tự chạy npm run capture trong Terminal của mình. Tôi cũng tự kiểm điều kiện tiên quyết: "bạn ktra eshop có đang run hok do chưa thấy terminal của eshop".

(c) Tôi tự chụp, và chụp lại khi ảnh sai. 5 ảnh trong resource-monitor/screenshots/ là tôi chụp tay theo mốc countdown của script, có ảnh phải chụp lại lần hai cho đúng cửa sổ. Rồi tôi đối chiếu chéo số của tool tự viết với số Activity Monitor tự đọc:

Lượt	sample-resources.sh (đỉnh)	Ảnh Activity Monitor	Đọc ra sao
Load	20,3%	13,4%	khớp bậc độ lớn
Stress	97,7%	91,7%	khớp — xác nhận node chạm trần 1 core
Spike	81,6%	34,9%	lệch — ảnh chụp trượt đỉnh → đã chạy lại, xem dưới
Soak	23,6%	16,3%	khớp bậc độ lớn

Ảnh luôn thấp hơn vì nó là một lát cắt tức thời, còn tool báo đỉnh của cả lượt.

Ảnh Spike thì trượt hẳn đỉnh sốc, và về sau tôi chạy lại lượt đó. Cần nói rõ ranh giới, vì "chạy lại để có ảnh đẹp hơn" đúng là việc tôi đã tự cấm mình ở trên: cái tôi làm không phải chụp lại một tấm rồi ghép vào lượt cũ, mà là chạy lại cả lượt (21:59, đủ 240s, 38.251 sample, 0% error) để .jmx · .jtl · dashboard · resources · ảnh cùng thuộc một lượt duy nhất. Ảnh mới đọc 72,6% so với đỉnh tool đo 75,7%. Và lượt 15:47 không bị xoá — nó thành lượt Spike thứ tư ở §3.4, nơi bốn lượt cho thấy node CPU đỉnh gần như nhau (75,7–81,6%) trong khi p95 đỉnh lệch 5,9 lần. Chọn một tấm ảnh trong cùng một lượt thì là chọn bằng chứng; chạy lại cả lượt rồi giữ luôn lượt cũ thì không.

(SV chưa tự kiểm) Bug `pgrep` bám vào tiến trình `zsh` bao ngoài thì tôi không bắt được — CSV vẫn có đủ cột, đủ dòng, chỉ là ghi RSS 0,5 MB. Hệ quả: file tài nguyên của lượt **Load** và **Stress** đầu **vô giá trị ở các cột `node_*`** (cột `java_*` vẫn đúng), và đã chạy lại cả hai sau khi sửa, thay toàn bộ `.jtl/dashboard/resources` theo đúng nguyên tắc ở lượt #5. - **Commit:** `test: sample CPU and RSS through each run`

Interaction #8 — Kiểm bug bằng request thật, và một kết luận bị bác bỏ

- **Task / Scenario:** bug report
- **Bước trong quy trình:** `jtl-analysis` bước 3 (phân loại phát hiện: kiểm được hay không)
- **AI tool:** Claude Code (Opus 5)
- **Date & time:** 2026-08-13 13:58
- **Prompt:** cùng lượt #4.
- **AI output (tóm tắt):** Viết `bug-report/verify-bugs.sh` chạy lại từng bằng chứng, rồi thực thi trên SUT đang chạy.
- **AI sai / bỏ sót: 9. Một "bug" AI khẳng định từ lượt #1 bị bác bỏ khi chạy thử.** AI đã nói `POST /api/admin/import-products` "báo số dòng đã insert nhỏ hơn thực tế" vì `stmt.finalize()` trả response trước các callback `stmt.run` ([server.js:234](#)). Kiểm bằng ba lô request thật: 5/5, 60/60, và 2/3 khi có một dòng thiếu `name` — **cả ba đều đúng**. `node-sqlite3` xếp các lệnh trên cùng handle theo thứ tự và callback của `finalize` chạy sau chúng. Kết luận sai này đã lan vào **5 file** (bug report, endpoint-selection, PLAYBOOK, SKILL perf-test-plan, chú thích trong `gen-test-plans.py` và `k6/workflow.js`) trước khi bị bắt.
- **Vì sao bỏ sót: phương pháp**, không phải model. Suy luận từ code nghe rất hợp lý và có kèm số dòng cụ thể, nên nó được ghi lại như một sự thật đã kiểm. Không có bước nào bắt phải phân biệt "đọc được từ code" với "đã chạy và thấy".
- **Human review: (SV đã kiểm)** Tôi không nhận `verify-bugs.sh` như một cái nút bấm — tôi hỏi nó **làm gì** trước ("*cái này là làm j*"), rồi đọc **toàn bộ output** và hỏi lại chỗ không hiểu ("*nó nó là sao*"). Chỗ tôi thấy thuyết phục nhất trong output không phải bằng chứng của bug mà là **dòng đối chứng**: `GET /api/orders/my-orders` không token vẫn trả **401**, trong khi `GET /api/orders/1` không token trả **200**. Không có dòng đó thì BUG-P1 chỉ là "endpoint này không cần token" — có thể bị phản biện là "API đó vốn công khai". Có nó thì thành **một route bị bỏ sót**, vì route ngay bên cạnh chặn đúng. Tôi cũng tự chụp [bug-report/screenshots/bug-evidence-verify-bugs.png](#) cho ảnh của Issue. **(SV chưa tự kiểm)** Cơ chế `node-sqlite3` xếp lệnh trên cùng handle — lời giải thích **vì sao** suy luận từ code lại sai — tôi nhận theo tài liệu chứ không tự dựng thí nghiệm riêng. Nhưng **kết luận** thì tôi kiểm được và nó đúng độc lập với cơ chế: 5/5, 60/60, 2/3 đều khớp DB. Đó cũng là lý do bug report **giữ lại** mục này ở phần "Ứng viên đã LOẠI" kèm bảng ba lô kiểm chứng thay vì xoá — một nhận định sai đã đi vào 5 file thì ghi lại vì sao nó sai có giá trị hơn là làm như chưa từng có nó.
- **Commit:** `docs(bug): verify each finding, and retract the one that failed verification`

Interaction #9 — Batch sạch bác bỏ phát hiện của chính lượt trước

- **Task / Scenario:** cả 4 scenario, batch 15/08
- **Bước trong quy trình:** `jtl-analysis` bước 2 (soát lỗi đọc metric) — lần này soát chính mình
- **AI tool:** Claude Code (Opus 5)
- **Date & time:** 2026-08-15 15:05
- **Prompt:** *làm đi (tiếp nối "okay sửa cho tui đi" — chạy lại batch sạch để `.jmx` khớp `.jtl` sau khi sửa lỗi #11)*

- **AI output (tóm tắt):** Batch 15/08 cho p95 Stress **7ms** — trong khi batch 14/08 cho **70ms** với cùng test plan. Kiểm `load_1m` trong file resources thì thấy nguyên nhân, và nó không phải nguyên nhân đã viết trong báo cáo.
- **AI sai / bỏ sót:** 12. **Kết luận nhân quả từ một so sánh không kiểm soát biến nào.** Báo cáo từng có mục §2.8 mang tên "*phát hiện quan trọng nhất của bài*": so hai batch, thấy p95 chênh 2,4–6,4 lần, kết luận **kích thước dữ liệu** là nguyên nhân, kèm cơ chế nghe rất hợp lý (SQLite một writer). Batch 15/08 bác bỏ: database **lớn hơn 16 lần** (50k → 830.139 dòng) mà **nhANH hơn ~10 lần**. Biến thật là `load_1m` — batch 13/08 chạy khi máy gánh 1,3–2 lần tải nền, nhất quán trên cả 4 lượt. Lúc đo vẫn có 4 container Docker, VS Code và một tiến trình AI agent chạy song song. Kết luận sai đó đã lan tới: **headline README**, **một nhánh flow chart Task 3**, và một dòng **self-assessment** khoe rằng bài tự chỉ ra lỗi hổng của chính nó bằng số đo.
- **Vì sao bỏ sót: phương pháp** — cùng loại lỗi #9 nhưng nặng hơn. Ở #9 tôi trình bày một suy luận từ code như sự thật đã kiểm. Ở đây tôi có **dữ liệu thật**, nhưng dữ liệu đó đến từ hai lượt chạy khác nhau ở **nhiều biến cùng lúc**, và tôi quy toàn bộ chênh lệch cho một biến tôi thấy thú vị. Một cơ chế đúng về lý thuyết (SQLite một writer) không chứng minh được nó là nguyên nhân của con số cụ thể này — hai việc khác nhau, và tôi đã trộn.
- **Human review: (SV đã kiểm)** Quyết định vào lượt này là của tôi, và nó là quyết định tốn công nhất của cả bài: sau lỗi #11 (nhãn "lockout" gán sai cho ~50.000 sample), tôi chọn **chạy lại cả batch** chứ không sửa nhãn trong `.jtl` cũ — để `.jmx` được nộp **đúng là file đã sinh ra .jtl được nộp**. Vá nhãn thì nhanh hơn nhiều nhưng bản nộp sẽ có một `.jmx` không khớp dữ liệu của nó, và tôi không muốn phải giải thích chuyện đó. Tôi cũng chốt: **§2.8 viết lại thành mục thu hồi, không xóa** — vì đó là nội dung giá trị nhất cho Task 2, một lỗi đọc metric thật, có bằng chứng, do chính bài này bắt được. Xóa đi thì bài trông sạch hơn mà mất đúng phần đáng đọc. **(SV chưa tự kiểm)** Tôi **không tự chạy lại** batch 13/08 để tái hiện điều kiện tải nền, và cũng không tự mở 4 file `resources-*.csv` đối chiếu cột `load_1m` — tôi nhận theo 4 cặp số trong §2.8. Nói cho đúng: điều tôi kiểm được là **kết luận cũ đã sai** (DB lớn hơn 16 lần mà nhanh hơn ~10 lần thì không thể do kích thước dữ liệu); còn `load_1m` là nguyên nhân thay thế thì với 4 điểm dữ liệu vẫn là **tương quan**, và §2.8 phải nói đúng như vậy chứ không được nâng lên thành nhân quả lần thứ hai.
- **Commit:** docs: retract the data-growth finding the new batch disproves

Interaction #10 — Tool của tôi in ra một con số dẫn tới kết luận sai

- **Task / Scenario:** Soak, batch 15/08 15:52
- **Bước trong quy trình:** `jtl-analysis` bước 4 (chốt endurance threshold)
- **AI tool:** Claude Code (Opus 5)
- **Date & time:** 2026-08-15 16:10
- **Prompt:** (tiếp nối lượt #9 — phần đọc kết quả sau khi batch cuối chạy xong)
- **AI output (tóm tắt):** `soak-drift.mjs` in Trôi RSS +228,9% cho lượt Soak, kèm kết luận "ỔN ĐỊNH" ở phần p95.
- **AI sai / bỏ sót:** 13. **Tool tự viết in ra một tỉ lệ so hai trạng thái khác nhau.** Trôi RSS được tính bằng **mẫu cuối chia mẫu đầu**, mà mẫu đầu (19,7 MB) lấy lúc process chưa warm-up xong. Ra **+228,9%** — đọc như một vụ rò rỉ bộ nhớ. Sự thật: RSS leo lên ~78 MB trong 60 giây ramp-up rồi **phẳng suốt 12 phút**; so nửa đầu với nửa sau chỉ **+5,0%** (74,5 → 78,3 MB). Nếu tin con số tool in ra thì báo cáo sẽ kết luận "có rò rỉ" trên một hệ thống hoàn toàn ổn định.
- **Vì sao bỏ sót: đặc điểm dữ liệu** — chuỗi đo có một giai đoạn warm-up ở đầu, và mọi phép so "đầu với cuối" trên chuỗi như vậy đều sai. Lỗi cùng họ với #12: so hai thứ không cùng loại rồi gọi kết quả

là "độ trôi". Khác #12 ở chỗ lần này biến gây nhiễu nằm **bên trong chính lượt đo**, không phải ở môi trường.

- **Human review: (SV đã kiểm)** Việc tôi tự làm ở lượt này là **báo ra một lỗi trong bằng chứng của chính mình**: script Soak gọi chụp hai mốc (phút thứ 1,5 và phút thứ 11,5) để cho thấy tài nguyên **không leo theo thời gian**, và tôi **quên mốc thứ hai** — nên tôi hỏi luôn "*này nó có kêu chụp ảnh thứ hai mà tui quên chụp có sao hok*" thay vì để đó. Kết luận sau khi cân: không chạy lại 12 phút, vì **cột node_rss_mb trong CSV đã đủ chứng minh phần "không leo"** còn ảnh chỉ là minh họa — nhưng phải **ghi rõ trong báo cáo là chỉ có một ảnh Soak**, không được im lảng để người đọc tưởng bộ ảnh đủ hai mốc. **(SV chưa tự kiểm)** Con số **+228,9%** thì tôi không bắt được, và nó là ví dụ đúng loại "AI sai mà không có gì báo lỗi": tool tự viết in ra một tỉ lệ có đơn vị hợp lý, đọc như rò rỉ bộ nhớ trên một hệ thống **hoàn toàn ổn định** (~78 MB phẳng suốt 12 phút, nửa đầu so nửa sau chỉ **+5,0%**). Tôi rút ra: **output của tool tự viết phải bị nghi ngờ đúng như output của SUT** — cả bài có 15 lỗi kỹ thuật thì **3 lỗi nằm trong tooling đo** (#1, #8, #13), tức ~1/5 lỗi đến từ dụng cụ đo chứ không từ hệ thống được đo. Đã sửa tool in **nửa đầu / nửa sau**, giữ lại dòng mẫu-đầu→mẫu-cuối kèm cảnh báo ngay dưới nó.
- **Commit:** `test(soak): compare like with like when reporting RSS drift`

Interaction #11 — Ba lần sinh viên bắt lỗi AI (chiều phát hiện đi ngược)

- **Task / Scenario:** §14 nội dung bản nộp · §2 "step by step" · cấu trúc repo
- **Bước trong quy trình:** soát lại toàn bài trước khi đóng gói
- **AI tool:** Claude Code (Opus 5)
- **Date & time:** 2026-08-15 16:40
- **Prompt:** ủa nội dung trong docs có nộp đầu dm Design and generate with AI — Nội dung xong · hỏi "step by step" / bạn nói hỏi kìa chỉnh lại đi vs lại có nên tạo folder task 1, 2, 3 để dễ dàng biết nội dung nó nằm ở task nào hok / mà phải đảm bảo là nó hok có vi phạm đề thì ms refactor lại
- **AI output (tóm tắt):** Ba lần đều phải sửa hoặc phải bảo vệ lập luận, không lần nào là "vâng làm ngay": (1) chuyển bằng chứng §5 từ docs/ vào report/main-report.md §1.1, (2) viết `design-log.md` ghi 7 bước quy trình, (3) **phản đối** việc refactor thành task1/2/3 kèm ba lý do, và bàn giao `ASKS.md` thay thế.
- **AI sai / bỏ sót:** Hai lỗi thật, cả hai do sinh viên bắt, không phải AI tự thấy.
- **(a) Bằng chứng §5 nằm ngoài bản nộp.** Bảng đăng ký endpoint của nhóm — thứ chứng minh "không trùng với 3 thành viên" theo §5 — chỉ có trong docs/endpoint-selection.md, mà docs/ **không nằm trong danh sách §14**. Tức là một yêu cầu có bằng chứng đầy đủ nhưng người chấm **không đọc được**, và theo §17 thì tài liệu thiếu bị **0 điểm** cho mục đó. Sửa: chép bảng vào main-report.md §1.1; package.sh ship thêm docs/endpoint-selection.md làm tài liệu phụ, PLAYBOOK và script video thì bỏ.
- **(b) Lỗi ở §2 "step by step" bị nói ra rồi để nguyên.** AI tự ghi trong báo cáo rằng prompt của sinh viên ngắn nên yêu cầu "step by step" chưa được đáp ứng đầy đủ — rồi **dừng ở đó**, coi việc thừa nhận là đã xử lý. Sinh viên chỉ ra: *thừa nhận một lỗi không phải là bịt nó*. Sửa: viết `design-log.md` ghi 7 bước — mỗi bước hỏi gì, căn cứ nào, quyết ra sao, đổi file nào, **bước nào bắt được lỗi nào** — kèm câu ranh giới nói rõ đây là **bước quy trình, không phải prompt của sinh viên** (trộn hai thứ lại là bịa, §11 cấm).
- **Vì sao bỏ sót:** cả hai là **phương pháp**, và cùng một gốc: AI kiểm "*nội dung có tồn tại không*" thay vì "*người chấm có đọc được nội dung đó không*". Lỗi (b) còn thêm một tật riêng — **thừa nhận hạn**

chế xong thấy đủ, vì câu thừa nhận nghe đã có vẻ trung thực.

- **Human review: (SV đã kiểm — cả ba đều do tôi khởi xướng.)** Câu hỏi §14 xuất phát từ việc tôi đọc danh sách nội dung bản nộp rồi so với chỗ tài liệu thật sự nằm — không so nội dung mà so vị trí. Về lỗi "step by step": tôi không nhận câu tự thừa nhận, vì một lỗi được ghi ra vẫn là một lỗi. Về refactor task1/2/3 : tôi **không** ra lệnh làm, tôi đặt điều kiện "*phải đảm bảo nó không vi phạm để thì mới refactor*" — và khi nghe ba lý do (§14 đòi **một** báo cáo chính chứa cả hai phần; copy sang folder task sẽ tạo **nguồn thứ hai** cho 30+ con số; dời results/ và test-plans/ làm hồng đường dẫn trong 13 script) thì tôi **bỏ ý định đó**. Chỗ này tôi tự đánh giá là quyết định đúng: gọn mắt hơn mà rủi ro lệch số liệu và vi phạm §14 thì không đáng đổi. **(SV chưa tự kiểm)** Tôi không tự dò lại **toàn bộ** 17 mục của đề để tìm những lỗi cùng kiểu "có nội dung nhưng nằm sai chỗ" — chỉ bắt được đúng hai chỗ này. TASKS.md là để bù cho việc đó.
- **Commit:** docs: move the §5 evidence into what actually gets submitted

Interaction #12 — Chạy flow chart Task 3 thật, và mắc lại đúng lỗi cũ lần thứ ba

- **Task / Scenario:** Task 3 §4 — hiện thực nhánh "PR pipeline" thành GitHub Actions
- **Bước trong quy trình:** sau khi tự chấm — bịt hai chỗ bị trừ điểm
- **AI tool:** Claude Code (Opus 5)
- **Date & time:** 2026-08-15 17:00
- **Prompt:** nếu bạn chấm 88 thì phải tự biết tại sao nó 88 rồi sửa lại cho uti sửa lại làm sao tự chấm nó lên 100 nha
- **AI output (tóm tắt):** Dựng [.github/workflows/perf-smoke.yml](#) (checkout SUT public → khởi động → seed qua API → sinh plan → JMeter non-GUI → cổng ngưỡng → artifact) và [tools/ci-gate.mjs](#). Chạy **5 lượt**, một lượt build **đỏ thật**. Ghi bằng chứng ở [ci/ci-runs.md](#).
- **AI sai / bỏ sót:** 16. **Rút kết luận về máy từ một lượt chạy — lần thứ ba của cùng một lỗi.** Hai câu đã **viết vào chú thích workflow và commit message** trước khi có điểm dữ liệu thứ ba: "*runner 2 vCPU sẽ chậm hơn nên lượt ngưỡng-8ms fail*" và "*p95 trên CI chậm hơn local 12,6 lần*". Chạy thêm hai lượt **cùng 20 VU, cùng plan, cùng runner spec, không khác một tham số nào** thì p95 ra **101 / 15 / 8 ms**. Tức: (a) ở 5 VU runner nhanh **bằng** máy 12 lỗi nên ngưỡng của local **PASS** và cho tín hiệu xanh giả; (b) chênh lệch giữa hai máy **nhỏ hơn phương sai của chính CI**, nên không quy cho máy được.
- **Vì sao bỏ sót: phương pháp**, y hệt #9 và #12. Ba lần cùng một hình dạng: có một quan sát, có một cơ chế nghe hợp lý (runner yếu hơn thì chậm hơn — đúng về lý thuyết), và **không có lượt thứ hai cùng cấu hình**. Điều đáng ghi là lần này lỗi xảy ra **ngay trong lúc đang viết mục Task 2 về đúng loại lỗi đó** — biết tên lỗi không ngăn được việc mắc nó; chỉ thêm một điểm dữ liệu mới ngăn được.
- **Human review: (SV đã kiểm)** Chỗ tôi can thiệp là **bắt phải sửa, không phải bắt phải giải thích**: tôi tự chấm bài 88 mà hai điểm trừ nằm đó không ai bịt thì trừ điểm để làm gì. Yêu cầu của tôi là "*phải tự biết tại sao nó 88 rồi sửa lại*" — và tôi kiểm được kết quả bằng thứ không cần tin lời ai: **có một lượt build ĐỎ thật** (31878612141) và bốn lượt xanh, xem được trong tab Actions. Một flow chart chưa từng chạy thì không có gì phân biệt nó với một hình vẽ đẹp. **(SV chưa tự kiểm)** Tôi không tự đọc lại từng dòng YAML của workflow, cũng không tự tính lại phương sai 12,6 lần từ ba file .jtl của CI. Nhưng tôi giữ nguyên yêu cầu về hình thức bằng chứng: repo bài làm là **private** nên link Actions người chấm **không mở được** → output cổng ngưỡng phải được chép nguyên văn vào [ci/ci-runs.md](#) trong repo, đúng cùng lý do như lỗi #14 (bằng chứng nằm ngoài bản nộp thì coi như không có).
- **Commit:** ci: run the Task 3 pipeline for real instead of describing it

Tổng hợp — AI sai gì trên toàn bài

#	L u ợt	AI sai / bỏ sót	Nhóm lý do (§6)	Đã sửa thành
1	#2	<code>preflight.mjs</code> đọc <code>java_home -V</code> từ stdout trong khi lệnh in ra stderr → báo <code>[OK]</code> với giá trị rỗng (lỗi im lặng)	đặc điểm công cụ	hàm <code>shell()</code> gộp <code>2>&1</code> , đọc <code>os.arch</code> trực tiếp từ JDK
2	#4	Nhánh <code>lockout</code> : <code>assertion regex</code> đúng nhưng không xoá được cờ <code>Fail</code> của <code>4xx</code> → smoke test báo 41,38% error trong khi SUT chạy đúng	đặc điểm endpoint	<code>JSR223PostProcessor</code> gọi <code>prev.setSuccessful(true)</code> cho <code>40[13]</code>
3	#4	Bước 5 <code>assert</code> cứng 200, bỏ qua state machine <code>FR-10</code>	đặc điểm endpoint	<code>assert 200 400 + reset-orders.mjs + orders.csv</code> chỉ dùng <code>confirmed</code>
4	#4	<code>orders.csv</code> xen kẽ <code>shipping</code> — chuyển đổi không hợp lệ từ <code>pending</code>	đặc điểm endpoint	chỉ sinh <code>confirmed</code> , sửa cả trong <code>seed-perf-data.mjs</code>
5	#5	<code>users.csv</code> chứa 2 dòng mật khẩu sai → luồng chính tự khoá tài khoản của mình, ~2,9% iteration vô giá trị	chất lượng prompt	tách <code>data/users_lockout.csv</code> , <code>users.csv</code> chỉ còn tài khoản hợp lệ
6	#5	Timer ở scope thread group → think-time chèn 5 lần/iteration, tải thực tế nhẹ hơn thiết kế ~5 lần	đặc điểm công cụ	chia lại 200–600ms mỗi bước để tổng đúng 1–3s
7	#6	Bước 5: 400 hợp lệ của <code>FR-10</code> vẫn bị tính là lỗi → báo 18,25% error trên hệ thống khoẻ. Cùng cơ chế đã sửa ở lỗi #2 nhưng không mang sang	suy luận không mở rộng	dùng lại <code>mark_expected_4xx("400")</code> cho bước 5
8	#7	<code>sample-resources.sh</code> bám vào tiến trình <code>zsh</code> bao ngoài thay vì <code>node</code> → ghi RSS 0,5MB / CPU 0% suốt lượt chạy	đặc điểm môi trường	lọc theo token đầu command line phải là <code>node</code>
9	#8	Khẳng định <code>import-products</code> báo sai số dòng insert — bác bỏ khi chạy thử (5/5, 60/60, 2/3 đều đúng). Đã lan vào 5 file	phương pháp: suy luận từ code trình bày như sự thật đã kiểm	giữ lại ở mục "đã loại" kèm bảng kiểm chứng; sửa lý do không <code>assert</code> theo <code>inserted</code>
10	#8	<code>Math.min(...array)</code> trong <code>summarize-jtl.mjs</code> tràn call stack với <code>.jtl</code> 264k sample — chạy qua bình thường ở lượt Load 16k	đặc điểm dữ liệu (chỉ lộ ở file lớn)	thay bằng <code>reduce</code>
11	#8	<code>mark_expected_4xx()</code> hardcoded chữ <code>"Expected lockout response"</code> , dùng lại cho bước 5 mà quên đổi → raw <code>.jtl</code> ghi nhận "lockout" cho ~50.000 sample của một endpoint không liên quan gì tới <code>lockout</code>	suy luận không mở rộng (cùng loại lỗi #7)	tham số hoá <code>reason</code> : bước 5 ghi <code>"FR-10 invalid transition"</code> , nhánh <code>lockout</code> ghi <code>"Account lockout"</code>
12	#9	Kết luận kích thước dữ liệu gây suy giảm 2,4–6,4 lần, từ một so sánh hai batch không kiểm soát biến nào. Batch 15/08 bác bỏ: DB lớn hơn 16 lần mà nhanh hơn 10 lần; biến thật là <code>load_1m</code> . Đã lan tới headline README, flow chart Task 3, self-assessment	phương pháp: nhân quả từ tương quan không kiểm soát	§2.8 viết lại thành mục thu hồi , kèm 4 cặp <code>load_1m</code> làm bằng chứng

#	L ư ợ t	AI sai / bỏ sót	Nhóm lý do (§6)	Đã sửa thành
1 3	#1 0	soak-drift.mjs tính "trôi RSS" bằng mẫu-cuối / mẫu-đầu, mà mẫu đầu lấy trước warm-up → in +228,9% , đọc như rò rỉ. Thực tế nửa-đầu/nửa-sau chỉ +5,0%	đặc điểm dữ liệu: chuỗi đo có warm-up ở đầu	so nửa đầu với nửa sau; giữ dòng cũ kèm cảnh báo
1 6	#1 2	Hai kết luận về lượt CI, rút từ một lượt mỗi kết luận: "runner 2 vCPU chậm hơn máy 12 lõi" (ở 5 VU nó nhanh bằng) và "CI chậm hơn local 12,6 lần " (lượt sau cho 8ms , đúng bằng local). Cả hai đã viết vào chú thích workflow trước khi có điểm dữ liệu thứ ba	phương pháp: nhân quả từ so sánh không kiểm soát biến — lần thứ ba , sau #9 và #12	chạy thêm 2 lượt cùng 20 VU → phương sai 12,6 lần giữa các lượt y hệt nhau; sửa chú thích + đổi hẳn đề xuất §4.3

| 17 | #12 | §3.4 ghi "ba lượt Spike cho p95 đỉnh **47 · 65 · 7ms**" rồi quy nguyên nhân cho load_1m . Đo lại từ .jtl : **47 · 8 · 12ms** — hai số sai; và load_1m **không xếp đúng thứ tự** (tải nền cao nhất 6,0 cho 12ms, lượt 47ms chỉ 4,5). Số 7ms còn tự mâu thuẫn với bảng cửa sổ 10s ngay phía trên (90–100s = 12ms) | **bản sửa không được mang sang chỗ khác**: §2.8 đã thu hồi việc quy nhân quả cho một biến đơn lẻ, nhưng §3.4 dùng đúng lập luận đó thì để nguyên — cùng họ #7/#11, đảo chiều | đo lại cả 3 .jtl + 3 file resources, ghi bảng thật; kết luận đổi thành "spike có phương sai lớn nhất, nguyên nhân **chưa biết**", hậu thuẫn bằng 3 lượt CI ở §4.4 |

(Số 14–15 nằm ở bảng "lỗi bản nộp" ngay dưới. Lỗi #16 và #17 là lỗi kỹ thuật nhưng phát hiện muộn nhất — lúc chạy CI thật và lúc soát lại số liệu Spike — nên chúng ở bảng này.)

Hai lỗi nữa, cố ý KHÔNG đánh số vào bảng trên, vì chúng không làm sai một con số nào — chúng làm sai **bản nộp**, tức một loại thiệt hại khác:

#	L ư ợ t	Lỗi	AI bắt đượ c	Đã sửa thành
1 4	#1 1	Bảng chứng §5 (bảng đăng ký endpoint của nhóm) chỉ nằm trong docs/ , mà docs/ không có trong danh sách §14 → yêu cầu có bảng chứng đủ nhưng người chấm không đọc được, §17 tính 0 điểm cho mục đó	sinh viên	chép bảng vào report/main-report.md §1.1 ; ship thêm docs/endpoint-selection.md
1 5	#1 1	Lỗi ở §2 "step by step" được AI tự ghi ra rồi để nguyên — coi việc thừa nhận hạn chế là đã xử lý hạn chế	sinh viên	ai-audit/design-log.md ghi 7 bước quy trình, kèm câu ranh giới với prompt của sinh viên

Tính cả hai thì con số đáng nói nhất của phụ lục này là: **2 trong 17 lỗi là do người bắt, không phải do AI tự soát ra** — và cả hai đều là loại lỗi mà **không script nào phát hiện được**, vì không có gì sai về mặt kỹ thuật để mà báo.

Nhận xét xuyên suốt: 12 trong 15 lỗi kỹ thuật **không làm test plan báo lỗi** — plan vẫn chạy, vẫn ra .jtl , vẫn sinh dashboard đẹp. Chúng chỉ làm con số **sai**. Đó là lý do bước "đọc kết quả trước khi tin nó" trong skill perf-test-plan không phải thủ tục hình thức: nếu chỉ kiểm "test có chạy không" thì cả 10 lỗi này đều lọt.

Chi phí thật của các lỗi: hai lượt chạy phải huỷ và xoá bỏ toàn bộ bằng chứng, cộng khoảng 25 phút chạy lại. Nếu không đọc .jtl giữa lượt mà chờ đến khi viết báo cáo mới đọc, thì cái phải làm lại là **cả**

bốn lượt cộng phần phân tích.

Bảng này chép sang [report/main-report.md §2.4](#) — viết một lần, dùng hai chỗ.